意卷 IntentForm

一句话需求 → 一份能发出去的卷子

参赛作品介绍文档

第九届传智杯全国大学生数智创新与 AI 应用大赛 · AI 创新应用挑战赛（技术应用赛 · 个人赛）

项	内容
作品名称	意卷 IntentForm：自然语言生成整份在线表单，发布、作答、判分、看板一条链
参赛形式	个人赛（单人完成，AI 助手协同产出代码，架构与验收由本人裁定）
代码仓库	https://github.com/Zeyu66666/intentform
证据版本	main 分支 9fa7475（2026-10-27 15:43），共 73 次提交，全部发生在 10-26 至 10-27 两天内；第 5 章截图是该分支后于 18:41 整轮重跑的版本
技术栈	TypeScript · Node 20+ · pnpm monorepo · React 18 + Vite 5 · SQLite（better-sqlite3）· Zod · KaTeX
本版说明	按官方「作品介绍文档」7 项要求重排：第 1–7 章逐项对应 (1)–(7)，技术深度与《技术文档》要求合并进同一份，附录给缺口与复现清单
自验命令	pnpm typecheck（4 个包）· npx vitest run（52 文件 / 566 用例）· pnpm lint（151 文件 0 告警） · node scripts/verify.mjs（VC1–VC4 4/4）· node scripts/browser-closed-loop.mjs（真浏览器 6 步 8 图）

阅读建议：评审若只想核一件事，看第 2 页的七项对照表 → 附录 B 的命令；若只怀疑一件事，看附录 A 的已知缺口——现在做不到的都写在里面，没有用「已完成」覆盖。

官方七项要求 · 本文对应位置

官方第 6 节对「作品介绍文档」提出 7 项硬性内容。下表逐项给出本文对应章节，以及该项能被核验的证据放在哪里。凡是写「见某 commit / 某脚本输出」的，都能在仓库里原样跑一遍。

官方要求	本文位置	该章用来交差的证据
(1) 项目详情及使用方式	第 1 章	六步闭环流程表、三条上手路径、四行启动命令；流程截图见第 5 章
(2) 项目亮点	第 2 章	六条亮点，每条都跟一句「用户省下什么」，并指向命令或文件
(3) AI 测试技术落地	第 3 章	566 个用例、两个 AI 生成用例提交、6 个被测试抓出的真实缺陷与修复 commit
(4) 关键技术	第 4 章	三大技术重点（流式管线 / 校验与修复 / 快照与版本）+ 选型与放弃清单
(5) 项目相关图片/照片	第 5 章	7 张同一次真浏览器运行落盘的截图（含手机视口），文件名与脚本步骤一一对应
(6) 是否发表或获奖	第 6 章	首次提交声明、未发表声明、第三方素材与开源许可说明
(7) 是否基于其他项目优化	第 7 章	自主开发声明与依据、开源依赖清单、AI 协同产出的边界
附录 A	已知缺口	8 条当前做不到的事，逐条写明影响与触发条件
附录 B	复现清单	五条命令、期望输出、证据索引表（截图 / fixture / prompt 链）
附录 C	四种模式怎么跑	评委自带 Key 时：live / record / replay / mock 四条命令、预期输出、坑与成本

文中数字的核对方式

数字	本文写法	一条命令自证
自动化用例	52 个文件 / 566 个用例全绿	npx vitest run
类型与静态检查	4 个包 typecheck 通过、oxlint 0 警告	pnpm typecheck；npx oxlint --deny-warnings
HTTP 级闭环	VC1–VC4 全部通过（4/4）	node scripts/verify.mjs
浏览器级闭环	六步全绿，截图落盘 8 张	node scripts/browser-closed-loop.mjs
提交数	73 次提交，每条带 [Pxx] / [AI-TEST] 标签	git rev-list --count HEAD

说明：本文同时承担官方 2.3「技术文档（≤30 页）」职责，故第 3、4 章写得更细；若提交页分两个上传栏，同一份文件分别上传即可。

1　项目详情及使用方式（对应要求 1）

1.1　给谁、解决什么

意卷 IntentForm 面向中小学与培训机构的一线教师。教师出卷的真实负担有三段：第一段是把脑子里的要求写成一份结构完整的卷子（题干、选项、答案、分值、解析）；第二段是把分值配平、题目 id 排重、问卷和判分题别混在一起；第三段是发出去之后还能不能收回来、分数对不对得上。现有问卷类工具把这三段全部留给人：它们要么要求人先手工搭好结构再填空，要么干脆只做收集不做生成。

本作品把第一段交给模型，把第二段交给机器校验，把第三段做成一条闭环：教师说一句需求（例如「初二二次函数随堂测，10 题，要有图像题，每题 2 分」），系统流式生成整份表单并逐字段上屏，逐行校验与整表门禁通过后一键发布成学生短链；学生在手机上作答提交，客观题当场判分，教师在看板看到这条提交并导出 CSV。

不做的事也写清楚：不做题库检索与题目查重、不做主观题自动批改、不做试卷排版打印、不做多教师协作。这些不是遗漏，是刻意把范围收在「一句话 → 一份能发的卷子 → 一份能对账的提交」上，范围外的能力一条都没有对外声称。

1.2　一条链的六步

步	谁操作	在哪个界面	这一步的产物
1 登录	教师	登录页（图 5-1）	会话 Cookie；密码用 argon2 校验，登录失败限流
2 生成	教师	三栏生成页（图 5-2）	SSE 事件流：阶段、思考摘要、逐字段落定、错误、修复、用量
3 发布	教师	发布面板（图 5-3）	一份不可变的已发布版本 + 学生短链 /f/<slug>
4 作答	学生	手机视口学生页（图 5-5、5-6）	只含公开字段的页面，不含答案、解析、难度标签
5 判分	系统	提交接口	客观题即时判分，返回得分与逐题对错，不回传答案
6 看板	教师	教师看板（图 5-4）	新增一行提交记录，可导出 CSV（表头固定，含得分）

这六步由 `node scripts/browser-closed-loop.mjs` 在真实浏览器里点一遍：脚本自起服务与 headless Chrome，每一步落一张截图到 docs/evidence/，跑完在标准输出打 JSON 摘要。最近一次运行（2026-10-27 15:43）六步全绿，判分 20/20，看板新增 1 行，八张截图同一次运行内连号。

1.3 三种上手路径

想干什么	怎么走	大约多久
只看生成效果	打开首页 → 点一个预置需求（初二数学 / 高一英语 / 高二生物）→ 开始生成；中栏逐字段出现，右栏时间线同步推进	30 秒
走完整闭环	生成完成后点「保存并发布」→ 拿到 /f/xxxxxxxx → 手机视口打开作答 → 提交 → 回 /dashboard 看这条提交与 CSV	3 分钟
只看数据不看生成	pnpm seed 后登录 → /dashboard，已有 2 份表单与 30 条提交（全部虚构数据）	1 分钟

1.4 本机运行（10 分钟复现）

```
pnpm install                    # Node >= 20.11, pnpm
pnpm seed                       # 重建 demo 库，打印演示账号与两条学生短链
pnpm dev:server                 # http://127.0.0.1:8787
pnpm dev:web                    # http://127.0.0.1:5173（必须用 127.0.0.1，不能用 localhost）
```

默认 LLM_MODE=replay，不需要任何 API Key、不需要联网。四种生成模式由服务端环境变量决定，前端顶栏显示实际生效的那一种：replay（默认，按「需求 + 模型 + prompt 版本 + schema 版本」的哈希命中录像逐 chunk 重放，未命中显式报错而不是静默造假）、mock（内置脚本，含一条故意写坏再修复的题，完全不出网）、live（真调上游，Key 只在服务端）、record（把上游 chunk 录成新 fixture）。

使用边界：服务端刻意不读 .env 文件（仓库里没有 dotenv），配置一律走进程环境变量，要落文件就用 node --env-file=.env 显式指定；需要 live 时把 Key 写在命令前缀上，不要提交进仓库。浏览器只验证过 Windows + Chrome 156，换环境要重跑闭环脚本后再引用证据。

2　项目亮点（对应要求 2）

六条亮点，每条都按「做法 → 用户因此省下什么 → 证据在哪」写。凡是没有证据支撑的，不进这一章（都进了附录 A）。

2.1　逐字段流式上屏，不是等一份大 JSON

常见做法是让模型吐一整段 JSON 再解析：题目一多、输出一长，中途一个字段写错就整份作废，用户只能盯着转圈然后重来。我们把生成结果按行切开，每落定一个字段就立刻上屏，右栏同时画出阶段与耗时。

用户省下什么：不用盲等两分钟才知道成败——第一道题在流开始后几秒就出现在屏幕上，觉得不对可以立刻取消（取消会中断上游请求，不继续烧 token）。

2.2　只修坏掉的那一行

逐行校验把错误精确到行，错误码与截断原文一起回给模型，只重生成坏行；修复最多两轮，且要求错误数单调变小，不收敛就停手并如实报失败。

用户省下什么：不会因为第 7 题少个答案就把前面 6 道好题一起丢掉；两轮上限也让等待时间有界，不会无限修下去。

2.3　整表门禁：单行合法不等于整卷可发

模型完全可能给出一份「每行都合法、整份不能发」的卷子：分值合计不是 100、两道题撞 id、问卷里混进判分题、判断题没给答案。发布前有一道整表门禁，六条判据全部通过才允许写已发布版本；不通过就不发学生链接，并把失败原因写在界面上。

用户省下什么：不用靠人肉复核总分和 id——把错卷发到班级群里再撤回，代价比多等两秒大得多。

2.4　发布即快照，改草稿不影响已发出的收集页

发布不是给表单打个 published 标记，而是写一份不可变的版本记录（form_versions 追加式）；学生短链指向某个具体版本，教师事后改草稿不会改变已发出页面的题目。学生若在换版后提交旧版，服务端返回 409 并提示刷新，而不是把旧答案记到新版本上。

用户省下什么：不会出现「学生答的是第三版、老师判的是第四版」这种对不上账的事故。

2.5 学生端零泄漏：答案不进 HTML

学生页只能经 publicView() 取数据，这个投影会删掉 answer、explanation、bloom、knowledgePoint 与 meta.objectives；渲染层还有第二道白名单（纵深防御，不是重复劳动）。提交响应只回得分与逐题对错，不回传答案。

用户省下什么：学生按 F12 也看不到答案——这是能不能拿去真用的底线，不是加分项。

2.6 确定性复现：没有 Key 也能跑通同一条链

三条回放录像 + mock 模式 + 两个自起服务的脚本（HTTP 语义闭环、真浏览器闭环）。评审不需要 API Key、不需要联网，就能在自己的机器上把「生成 → 发布 → 作答 → 判分 → 看板」跑一遍，并拿到同样的截图与同样的 JSON 摘要。

用户省下什么：「我这边能跑」不再是口头承诺——评审可以自己验证，而且结果可重复。

亮点	证据（可点开）
2.1 逐字段上屏	web/src/stream/（状态机与解码）、图 5-2 三栏生成页、时间线面板
2.2 只修坏行	packages/llm/src/repair.ts、commit 318d89c（把「缺分值」提前到行级，才修得动）
2.3 整表门禁	packages/schema/src/rules.ts（R1–R8）、verify.mjs VC4、11 个错误码
2.4 发布快照	verify.mjs VC1（改草稿不影响已发布收集页）、VC2（旧版提交 409）
2.5 零泄漏	packages/schema/src/public-view.ts、commit 04c0e6f、verify.mjs VC3
2.6 可复现	fixtures/replays/ 三组录像、mock 模式、两个 scripts/*.mjs

3　AI 测试技术落地使用情况概述（对应要求 3）

这一章只写已经发生的事：跑了多少、谁写的用例、抓到什么缺陷、哪一条修好了、哪一条还没做到。不写「计划中」当「已完成」，也不写没有测量过的覆盖率百分比。

3.1　当前真实的测试规模

检查项	真实结果	什么时候跑的
单元 / 集成测试	52 个测试文件、566 个用例全部通过，2.11 秒	2026-10-27 18:36（npx vitest run）
类型检查	4 个包 tsc --noEmit 全部 0 错	2026-10-27 18:37（pnpm typecheck）
静态检查	oxlint --deny-warnings：151 文件，0 warning 0 error	2026-10-27 18:37（pnpm lint）
HTTP 语义闭环	VC1 快照隔离 / VC2 判分与 409 / VC3 零泄漏 / VC4 数据对账，4/4 通过	2026-10-27 18:36（node scripts/verify.mjs）
真浏览器闭环	6 步全绿，八张截图，判分 20/20，看板新增 1 行（slug s5xezyggjy）	2026-10-27 15:43（browser-closed-loop.mjs）
契约测试	SSE 7 类事件 / 递增 seq / 心跳不算事件、NDJSON 逐行、publicView 白名单	随每次 vitest 运行
回放录像	3 组（33 / 27 / 27 个 chunk），带 meta.json 记录模型、prompt 版本与用量	fixtures/replays/

口径说明：上面每一行都是当轮命令的真实输出，不是累计统计。上一轮（10-27 08:23）的用例数是 114 条 web 用例的子集，本轮总数从 566 起算；如果评审在自己机器上跑出不同的数字，以命令输出为准，本文不追认任何未跑过的数。

3.2　AI 在测试里具体做了什么

(a) 生成边界与脏输入用例。提交 9f240a0 `test(schema): AI-generated validator cases [AI-TEST]`：让 AI 针对校验器生成三类容易漏的用例——上限边界（差一位 / 正好 / 超一位，如 optionsMin=2、fieldsMax=100、titleMax=500）、组合场景（只有分节的 quiz、空 items、survey 带 totalScore）、脏输入不崩（数组、数字、字符串、未知键）。共 17 条，逐条人工核对期望值后保留；其中 2 条以「已知缺口」命名，断言的是「现在的行为」，规则补上时必须一起改 —— 这样缺口不会被忘记，也不会被写成已支持。

(b) 生成配置矩阵用例。提交 f218e40 `test(llm): cover mode parsing and api base resolution [AI-TEST]`：覆盖 4 种 LLM 模式 × 缺失 / 非法 / 合法三种取值，共 14 条。非法值必须抛错而不是静默降级，这条纪律由这批用例守住。

(c) 用 AI 起草契约测试，人工裁定口径。SSE 事件契约（7 类事件、递增 seq、心跳 15 秒一次且不算事件、断流要有终态）先被写进 schema 包的契约文件，再由 AI 起草断言、本人删掉其中「把实现细节当契约」的部分。判据是：这条断言失败时，是接口变了还是实现错了？答不上来的断言不收。

(d) 用 AI 做缺陷定位，但只采纳能被验证的假设。真实做法是：把失败输出、相关文件与最近的改动一起给 AI，要求它给三个按代价排序的假设，然后逐个用最小复现去否掉。docs/prompt-log/P02、P04 的「AI 修改建议」一节记录了当轮被否决的建议（例如把 seq 跳号直接当断流处理），以及为什么否决。AI 提议的修复不经过回归用例，不算修完。

AI 在测试里的角色是「扩用例、提假设」，不是「下结论」：期望值由人核对，缺陷是否关闭由回归用例说话。

3.3 被测试抓出来的 6 个真实缺陷

下面每一条都真实发生过，都有 commit 可查、都有回归断言。它们同时也是「测试有没有用」的证据：如果测试只是复述实现，这 6 个缺陷一个都抓不到。

#	现象（用户能感知的）	谁抓到的	修复 commit
1	服务端起不来：node 直跑 TS 时相对导入缺扩展名	启动自检（server 单测前的冒烟）	6315c9f fix(schema): 补 .ts 扩展名
2	学生页 HTML 里能翻到难度标签与教学目标（公开视图漏字段）	契约测试 + verify.mjs VC3	04c0e6f fix(schema): publicView 剥 bloom/knowledgePoint/objectives
3	mock 模式声称走了修复环，实际是预置好的假流程	端到端用例（不是单测）	45c411a fix(llm): mock 真跑修复环
4	题目缺分值只在整表阶段报错，修复环拿不到行号，修不动	整表门禁用例	318d89c fix(llm): 缺分值提前到行级
5	单选题点得动、存得下，界面上不亮（选中态算法只认数组）	真浏览器闭环脚本（截图里看见的）	f63613d fix(web): 单选选中态可见
6	闭环脚本偶发失败：Windows 句柄延迟导致临时目录删不掉	闭环脚本连续运行	3def01f fix(scripts): 容忍句柄延迟

第 5 条值得单独说：它是被「截图」抓到的，不是被断言抓到的。I4 的验收只断言了「能提交、无泄漏」，没断言选中态，所以这个缺陷一直活到浏览器闭环那一步。修完之后补的是断言（选中态必须与状态一致），不是补一次手工点击。这件事直接改变了本项目的测试策略：凡是有视觉状态的交互，断言必须落到「渲染结果」而不是「状态对象」。

3.4 没做到的部分（同章交代，不藏到附录）

- 没有统计行覆盖率：仓库没装 c8 / istanbul，本文不给任何覆盖率百分比。说「覆盖率高」而没有测量，就是编。
- 没有对 live 模型输出做质量评估：整份需求达标率（30 条需求的分母口径）尚未完成，属于省赛阶段的工作。
- 三条回放录像录自本机 OpenAI 兼容 stub，meta.json 的 upstream 字段如实写着 local-stub；真实上游重录要等有 Key。
- 只在 Windows + Chrome 156 验证：没有跨浏览器矩阵，也没有 Linux / macOS 的实跑证据。
- E2E 只有一个浏览器脚本（一条主链），没有覆盖分支场景，例如生成失败的降级、发布面板的异常路径。

3.5 怎么自己验一遍

```
pnpm typecheck && npx vitest run && pnpm lint
node scripts/verify.mjs              # 期望：VC1..VC4 全部通过，exit 0
node scripts/browser-closed-loop.mjs # 期望：六步截图落盘 + stdout 打 JSON 摘要（需本机 Chrome）
```

verify.mjs 不碰 data/intentform.db，会自己起临时服务与临时库再关掉；browser-closed-loop.mjs 用 CHROME_PATH 指定浏览器，找不到会 exit 2 并列出找过的路径——不猜、不静默跳过截图。

4　项目中涉及的关键技术（对应要求 4）

三个技术重点，每一个都用「真实反例 → 错误假设 → 决策 → 修复 → 回归」讲，而不是罗列名词。每条后面附一句「为什么不用现成方案」。

4.1　重点一：流式管线的三层边界

模型输出是字节流，落到界面上是结构化的字段，中间有三层边界必须都处理对，任何一层偷懒都会在真实需求下翻车：

- 第一层，行切分：上游 SSE 的 chunk 边界与行边界无关，一个 chunk 可能切在半个汉字中间（UTF-8 多字节被截断），也可能什么都不含只是心跳注释。
- 第二层，JSON 增补：一行是一个完整的题目对象，但它可能是流里最后一个 token 才收尾——末行没有换行符，必须由结束事件兜底补齐，否则最后一题永远不出现。
- 第三层，校验上屏：字段落定即校验，错误带行号与错误码回给修复环；界面只渲染通过校验的字段，坏行显示占位与原因，而不是静默消失。

反例：最早的实现把「末行无换行」当异常丢弃，结果一份 10 题的卷子界面只显示 9 题。这个错误只有跑真实流才会暴露，用假的分块数据永远测不到——所以回放录像必须保留真实时序（chunk 大小与间隔），这也是 fixtures/replays 用 ndjson 而不是 JSON 数组的原因。

4.2　重点二：逐行校验 + 整表门禁 + 有界修复

校验分两级，职责不同：行级规则 R1–R8 判「这一行是否成立」，整表判据判「这一份能不能发」。两级用同一套错误码（11 个），错误对象带 path、message 与 repair 建议，所以同一条错误既能渲染到界面，也能直接拼进修复用的 prompt。

层级	判什么	典型错误码	什么时候执行
行级	答案在选项内、多选合法、选项数量、id 格式、LaTeX 可解析、难度标签合法	E_ANSWER_NOT_IN_OPTIONS / E_MULTI_ANSWER_INVALID / E_OPTIONS_INVALID / E_LATEX_INVALID	每个字段落定时
整表	分值合计 = 100、id 全局唯一、问卷不得含判分题、判分题必须有分值、字段数上限	E_SCORE_SUM / E_DUP_ID / E_SURVEY_HAS_ANSWER / E_ANSWER_MISSING	流结束时、发布前

修复环的纪律是三条：只修报错的行（不整份重发）、最多两轮、每轮的错误数必须严格变小。三条都不满足就停下并报失败，把「修不动」当结果返回给用户，而不是继续烧 token 假装在工作。commit 318d89c

就是把「判分题缺分值」从整表提前到行级，否则修复环拿到的错误没有行号，根本修不动——这是「先修可定位性，再修正确性」的一次真实取舍。

为什么不用现成方案：表单层没有引入 formily / amis / SurveyJS / react-jsonschema-form。它们的校验是「提交时校验表单值」，而这里要校验的是「模型刚生成的结构是否可发布」，判据、时机、错误语义都不同；套上去只能把规则写进配置里伪装成校验，反而更难解释。

4.3 重点三：快照与版本一致性

数据模型只四张主表：teachers、forms、form_versions（追加式）、submissions。发布 = 在 form_versions 里追加一条不可变版本并让短链指向它；提交 = 带着 versionId 落库。这样三种不一致都被堵住：改草稿不影响已发布页；旧版页面提交被 409 拒绝；同一个 slug 的历史提交永远能对上是哪一版。

对应的测试：VC1 改草稿后已发布收集页内容不变；VC2 判分正确且旧版提交返回 409；VC4 生成→发布→提交→看板的数据对账（得分、行数、CSV 表头）。这三条在 verify.mjs 里跑的是真 SQLite、真 HTTP、真判分，不是打桩。

4.4 技术选型与理由

层	选了什么	为什么	刻意没选
结构契约	Zod 定义 + z.infer 派生类型 + DeepReadonly	一份定义同时给运行时校验与编译期类型，避免两处漂移；publicView 的删除逻辑天然可测	JSON Schema + 代码生成（多一层构建产物）
服务端	node:http 零框架 + better-sqlite3 + argon2	只用预编译 SQL、请求路径全在掌控内；依赖越少，评审复现时踩坑越少	Hono / Express / Nest（本轮不需要中间件生态）
生成层	手写 OpenAI 兼容 Provider + 手写 SSE 读取 + NDJSON 行切分	四种模式（live/record/replay/mock）需要同一份时序语义，SDK 会把这层藏起来	LangChain / LlamaIndex / Vercel AI SDK
前端	React 18 + Vite 5 + 手写设计 Token	生成页的核心是状态机（乱序缓存、跳号、取消），不是组件库能力	formily / amis / SurveyJS / react-jsonschema-form
公式	KaTeX（trust:false）	题干含公式是刚需；trust:false 关掉 \href 一类能力，避免 XSS 面	MathJax（体积与首屏）
质量	vitest + oxlint + tsc --noEmit + 2 个自起服务的脚本	要在没有 Key 的机器上也能跑通，脚本必须自包含	Playwright / Cypress（依赖下载重，复现成本高）

4.5 约束是怎么落地的

AI 能写代码的前提是边界被写死。仓库根目录的 AGENTS.md 把四件事钉住，任何一次改动都要过：

- 模块边界：schema 不许依赖 Node 与浏览器；llm 只依赖 schema 的类型；server 依赖 schema + llm；web 只依赖 schema 的类型与纯函数，拿不到任何密钥。
- 协议不可改：7 类事件与递增 seq、心跳 15 秒是注释行不算事件、NDJSON 逐行、修复 ≤2 轮且单调、回放键与 fixtures 目录约定、字段 id 规则、错误码以附录为源。
- 类型纪律：禁 as any 与 @ts-ignore；跨包只走包名导入；契约先行（先提交类型与载荷，再写实现）。
- 提交纪律：commit-msg 钩子强制 feat / fix / test 必须带 [Pxx] 或 [AI-TEST] 标签，保证 73 次提交能追到具体迭代与对话链。

4.6 关键技术落在哪个文件

上表讲的是「为什么这么做」，下表是可以直接按图索骥去读代码的索引：每条技术点在仓库里的确切位置，以及它在七项要求里服务于哪一项。

技术点	代码位置	一句话说明	对应要求
结构事实源	schema/src/types.ts、rules.ts	Zod 定义同时产出运行时校验与编译期类型；R1–R8 是行级规则	1、4
公开投影	schema/src/public-view.ts	白名单式删字段，学生端响应体里没有 answer / explanation	2
SSE 契约	schema/src/sse-events.ts	7 类事件与递增 seq 由两端共用一份定义，不靠口头约定	1、4
上游适配与校验	llm/src/provider.ts、sources.ts、validate-line.ts	手写 OpenAI 兼容读取；四种模式共用一份时序语义；行级错误带错误码	4
整表门禁与修复环	llm/src/publish-gate.ts、generate.ts、repair.ts	发布前 6 条判据；不通过不发 done；修复只动报错行且必须严格变小	2、3、4
快照与判分	server/src/forms-route.ts、submissions-route.ts、grading.ts	发布 = 追加不可变版本；旧版提交 409；得分回填看板	2
前端状态机与复现脚本	web/src/stream/generate-machine.ts、scripts/*.mjs	乱序缓存、跳号兜底都在状态机里；两条证据脚本评审可自己跑	1、3、5

5　项目相关的图片/照片（对应要求 5）

全部图片来自同一次真浏览器运行（node scripts/browser-closed-loop.mjs，2026-10-27 18:41），由脚本在每一步自动落盘到 docs/evidence/，文件名前缀就是步骤序号，没有手工摆拍、没有事后补图。页面里的数据来自 mock 模式的内置脚本（含一条故意写坏再修复的题），教师账号与提交记录由 pnpm seed 生成，全部虚构。



图 5-1 / 5-2　左：登录页（argon2 校验 + 失败限流）；右：三栏生成页（需求 / 逐字段上屏 / 时间线）



图 5-3 / 5-4　左：发布面板（门禁通过才能发布，给出学生短链）；右：教师看板（新增一行，20 / 20）



图 5-5 / 5-6 / 5-7　手机视口三态：未作答 → 作答中 → 提交成功 20 分（无答案、解析、难度标签）

另有一张 03b-published-dashboard.png（发布后回到看板的中间态）在仓库里，为避免重复未嵌入。七张图连号同源，可对照 docs/evidence/README.md 的「最近一次运行」表核验时间戳。

6　项目之前是否发表过或获奖（对应要求 6）

项	实际情况
是否发表过	未发表。没有论文、没有公开文章、没有在任何应用商店或开源平台发布过成品
是否获奖	未获奖。本项目为首次参赛作品
是否首次提交	是。仓库的 73 次提交全部发生在 2026-10-26 至 10-27，为本次比赛新建，无历史版本对外发布
与其他赛事的关系	此前未以任何形式（含课程作业、毕业设计、其他赛事）提交过同一作品或其主要模块
预指导与省赛的关系	本版用于预指导；按组委会说明，预指导作品不作为省赛最终作品，省赛提交时以届时版本为准
第三方素材	未使用任何商用字体、图片、音频、视频素材；界面为自制，图标用系统字符与 CSS 绘制
演示数据	全部为种子脚本生成的虚构数据（2 份表单、30 条提交、虚构教师账号），不含任何真实师生信息

7　若在其他项目基础上优化，请说明优化部分（对应要求 7）

声明：本项目完全自主开发，未在任何其他项目、开源仓库、课程模板或商业脚手架的基础上优化。没有 fork 记录，没有克隆他人仓库后改名，也没有把别人的项目拆开重贴。

三条可核验的依据：

- 仓库第一次提交是 chore: init pnpm monorepo，此后 73 次提交按迭代 I0–I5 推进，每条都能在 git log 里看到功能与测试成对出现；不存在一次性倒入大量代码的提交。
- 表单渲染、校验规则、SSE 协议、发布快照、判分逻辑全部是本仓库自写；被明确排除的框架（LangChain / LlamaIndex / Vercel AI SDK / formily / amis / SurveyJS / react-jsonschema-form）一个都没有进依赖树。
- 第三方依赖只有通用基础库，清单与用途如下（全部为 MIT / Apache-2.0 类宽松许可，未修改其源码）。

依赖	用途	许可
React 18 / React DOM	界面渲染	MIT
Vite 5 + @vitejs/plugin-react	开发与构建	MIT
Zod	结构定义与运行时校验	MIT
KaTeX	公式渲染（trust:false）	MIT
better-sqlite3	本地数据库驱动	MIT

@node-rs/argon2	密码哈希	MIT
vitest / oxlint / typescript	测试、静态检查、类型检查	MIT / Apache-2.0
tailwindcss	样式工具链（仅生成原子类，未用其组件库）	MIT

与 AI 协同产出的边界（这一条按实写）：代码由本人与 AI 助手协同产出，AI 负责按契约实现与扩用例，本人负责架构约束（AGENTS.md）、协议冻结、验收与提交。因此本作品既不是「复制他人项目」，也不是「AI 一键生成」——每一次提交都能追到一条对话链（docs/prompt-log/P00–P04），且每条链都记了被否决的 AI 建议。评审若要求核实，按附录 B 的命令与目录即可。

最后说明本文档自身的变化：上一稿《参赛说明》按技术专题组织（10 页），本稿按官方 7 项重排并新增第 3 章（AI 测试）与第 5 章（截图），技术内容不删只挪，避免与《技术文档》重复描述同一条链。这是文档的优化，不构成项目的继承关系。

附录 A　已知缺口：现在做不到的，写清楚

这一节的作用是「评审怀疑某功能没做时先看这里」。下列 8 条都不是计划，是当前版本的真实状态；每条写明影响范围与触发条件，避免读者把它当成「以后会有」。

#	缺口	影响 / 触发条件
1	不做 Patch：改表单只能整份重发	教师想改一道题也要重新生成或整份保存；旧答案不与新版本合并
2	版本冲突只有 HTTP 级证据（VC2），没有浏览器级点击证据	界面侧冲突分支有单测，但缺一次真实点击；需等 Patch 上线才能补
3	没有模型输出质量评估（整份需求达标率）	只能证明链路通电，不能证明「模型答得好」；属省赛阶段工作
4	三条回放录自本机 OpenAI 兼容 stub	meta.json 的 upstream 为 local-stub；真实 DeepSeek 输出待有 Key 后重录
5	只在 Windows + Chrome 156 验证	换浏览器或系统需重跑闭环脚本，否则截图证据视为过期
6	限流是进程内计数，不是 IP 级分布式限流	单机演示够用；多实例部署会失效
7	CSV 未加 BOM	Excel 直接双击打开中文可能乱码，需另存为 UTF-8 或从数据导入
8	没有跨浏览器与移动端真机矩阵	手机截图来自 Chrome 的 390×844 视口模拟，不是真机

附录 B　复现清单与证据索引

五条命令覆盖「能不能跑」与「能不能对上」两件事，按顺序执行即可，不需要 API Key：

```
pnpm install
pnpm typecheck                          # 4 个包，0 错
npx vitest run                          # 52 文件 / 566 用例
node scripts/verify.mjs                 # VC1–VC4，4/4
node scripts/browser-closed-loop.mjs    # 6 步 8 图（需 Chrome，找不到 exit 2）
```

期望输出（verify.mjs）：

```
[verify] VC1 发布快照：改草稿不影响已发布版本的收集页 … 通过
[verify] VC2 判分与版本冲突：得分正确 / 旧版提交 409 … 通过
[verify] VC3 学生端零泄漏：响应体不含答案类键 … 通过
[verify] VC4 生成→发布→提交→看板：SSE 组装、得分对账、CSV 表头 … 通过
[verify] 全部通过（4/4）
```

证据	位置	能证明什么

闭环截图 8 张	docs/evidence/*.png（连号，同一次运行）	界面真的能用：六步点得通，不是只有接口
截图与版本对应	docs/evidence/README.md 的「最近一次运行」表	截图属于哪次运行、哪个 commit、什么浏览器
HTTP 语义证据	scripts/verify.mjs（VC1–VC4）	快照隔离、409 冲突、零泄漏、数据对账
回放录像 3 组	fixtures/replays/*.ndjson + *.meta.json	无 Key 可重放真实时序；meta 记录模型与用量
Prompt 链 5 条	docs/prompt-log/P00–P04（核心三条 P01/P02/P04）	每条链含提问、AI 建议（含被否决项）、修 bug 过程、commit
架构约束	AGENTS.md（模块边界、协议冻结、提交纪律）	AI 生成的代码为什么没有长歪
测试与类型	52 个 *.test.ts、4 个 tsconfig、oxlint 配置	566 用例、0 类型错、0 告警

本文档由 docs/deliverables/build_intro.py 生成（python-docx），改字改脚本重跑，不手改 docx，以保证文中引用的数字与仓库状态可复现。

附录 C　四种模式怎么跑（评委自带 Key 时）

模式由服务端的环境变量 LLM_MODE 决定，前端只是消费方——所以换成评委自己的 Key 不需要改一行代码、不需要重新构建前端。启动后 GET /healthz 会直接回当前模式，一眼可辨，不用猜：

```
curl -s http://127.0.0.1:8787/healthz
{"ok":true,...,"llmMode":"replay","needsApiKey":false}
```

模式	联网	Key	写盘	拿来干什么	耗时量级
replay（默认）	否	不需要	只读 fixtures/replays/	演示、评审复现、录视频	instant 下 < 1 秒
mock	否	不需要	不落盘	单测、确定性截图（内置一条写坏再修复的题）	即时
live	是	必需	不落盘	评委用真 Key 看真实模型输出	10–60 秒（受上游与超时约束）
record	是	必需	写 <hash>.ndjson + .meta.json	把一次真实调用录成回放，之后零 Key 可重放	同 live

四条命令（Windows 的 bash / Git Bash；PowerShell 把前缀换成 $env:LLM_MODE="live"）。四个端口故意错开，方便对着开、来回切：

C.1　replay：不需要 Key，默认就是它

```
PORT=8787 LLM_MODE=replay node --experimental-strip-types server/src/main.ts
# 期望启动日志：llm mode=replay（不打印 Key 相关提示）
# /healthz -> "llmMode":"replay","needsApiKey":false
```

三条预置需求（初二数学 / 高一英语 / 高二生物）能完整重放；换个说法就是未命中，而 replay 不会伪造结果——下面是实测的报文（不是示意图）：

```
# 用一条录像里没有的需求
curl -s -N -X POST http://127.0.0.1:8787/api/generate \
  -H 'content-type: application/json' \
  -d '{"requirement":"不在录像里的需求"}'
event: error
data: {"type":"error","seq":6,"code":"llm_config",
       "message":"回放未命中：本机没有这条需求的录制文件。replay 不伪造结果……",
       "retryable":false}
```

C.2　mock：不联网、不落盘，用来演示与截图

```
PORT=8788 LLM_MODE=mock node --experimental-strip-types server/src/main.ts
# 期望：任何需求都吐同一份确定性表单（6 个节点、20 分），其中 1 条故意写坏、被修复环改对
# 时间线上能看到「q_repair_demo（第 3 项）已修复」+ 定向修复 1 轮
```

第 5 章那七张截图的生成步骤就是 mock——只有它能让「同一份需求每次得到同一份卷子」，截图才可复现。

C.3 live：评委填自己的 Key，真调上游

```
PORT=8789 LLM_MODE=live \
  LLM_API_KEY=sk-评委自己的 key \
  LLM_BASE_URL=https://api.deepseek.com/v1 \
  LLM_CHAT_MODEL=deepseek-chat \
  node --experimental-strip-types server/src/main.ts
# 期望：/healthz -> "llmMode":"live","needsApiKey":false
# 期望：生成时中栏逐字段上屏，右栏时间线出现真实 usage（prompt / completion tokens）
```

没有 Key 就跑 live 会在启动期直接崩（实测输出），不会拖到第一次生成才报错：

```
$ PORT=8799 LLM_MODE=live node --experimental-strip-types server/src/main.ts
LlmError: live/record 模式需要 LLM_API_KEY
  code: 'llm_config',
  detail: 'LLM_API_KEY is empty; replay/mock 不需要 Key'
```

Base URL 是 OpenAI 兼容协议，换模型服务商只改 LLM_BASE_URL 与 LLM_CHAT_MODEL 两项；超时 LLM_TIMEOUT_MS（默认 60000）、单次上限 LLM_MAX_TOKENS（默认 4096）、温度 LLM_TEMPERATURE（默认 0，生成链路固定贪心）；生成接口另有 RATE_GENERATE_PER_MIN（默认 10）的限流。

C.4 record：把一次真实调用录成回放，之后零 Key 可重放

```
PORT=8790 LLM_MODE=record \
  LLM_API_KEY=sk-评委自己的 key \
  node --experimental-strip-types server/src/main.ts
# 期望：生成结束后 fixtures/replays/ 多出一对文件
#   <replayKey>.ndjson     上游 chunk 的真实时序（{d, dt}）
#   <replayKey>.meta.json 模型、prompt 版本、chunk 数、总耗时、usage
```

两条纪律写死在代码里，评委会碰到：①只在正常结束时落盘，中途断流不写半截文件（半截 fixture 会被读侧判成损坏，比没有更糟）；②回放键由「需求 + 模型 + prompt 版本 + schema 版本」哈希而成，同一需求重录会覆盖同名文件。当前仓库里的三条录像，meta 里记的是 local-stub 上游与 deepseek-chat，也就是说它们来自联调桩，不是真实 DeepSeek 输出——这一点在附录 A 已列，评委用 record 重录即可换成真货。

C.5 四个坑（都是真实踩过的）

- 服务端不读 .env：cp .env.example .env 不会生效，必须用 --env-file=.env 或把变量写在命令前缀上（仓库里没有 dotenv）。
- 换端口要同步改 APP_ORIGIN，否则 CORS + CSRF 来源校验直接 403；浏览器也用 127.0.0.1，别用 localhost。
- LLM_REASONING_MODEL 只出现在 .env.example 里、代码尚未读取——别把它当开关用（附录 A 已列）。

- Key 只在服务端，前端产物里没有；.env 与 .env.* 已 gitignore，最稳是放仓库外再用 --env-file 指过去。

一条命令自证模式确实切了：curl -s http://127.0.0.1:8788/healthz 只看 llmMode 字段；界面顶栏同时显示当前数据源，页面里的人不会误以为真模型在跑。

C.6 按你想验的东西选一条

你想验什么	怎么做	期望看到
只看界面，不配任何 Key	LLM_MODE=replay + pnpm dev:web，点预置需求	中栏逐字段上屏、右栏时间线推进、发布后才出现学生短链
真模型能不能写卷子	LLM_MODE=live + 自己的 Key	时间线出现真实 usage（prompt / completion tokens），题目内容随需求变
换模型或换服务商	改 LLM_BASE_URL 与 LLM_CHAT_MODEL 两项	启动日志与 /healthz 的 llmMode 变 live，生成照常
留一份自己的录像给评审	LLM_MODE=record 跑一次后改回 replay	fixtures/replays/ 多一对 ndjson + meta，之后零 Key 可重放
确认现在到底是哪个模式	curl /healthz 或看页面顶栏	"llmMode" 与界面显示一致，不会含糊
核对判分与数据	生成→发布→作答→提交后看 /dashboard	提交行得分与 CSV 表头一致（本次闭环 20 / 20）

一句话：评委不需要动代码，也不需要任何 Key 就能看完整条链；想验证「真模型也行」，四条命令里的第三条就够。